NFsim Complexation — how the Step actually executes the biology

`flagella_nfsim_complexation.py` — `ecoli-flagella-nfsim-complexation`, wired into `ecoli_baseline.py` (opt-in feature, not yet default)

One new Vivarium Step replaces four deterministic assembly Steps:

- **FlagellaNFsimComplexation** — wraps NFsim/BioNetGen, a rule-based, network-free stochastic engine. Still Gillespie-style timing (propensity-weighted, exponential waits), but species are individual tracked graph objects with real bond/site state, not bulk counts.
- Fires on a **rate-limited cadence (~1200s)**, not every tick — spawning a BioNetGen subprocess every 2s tick would be far too slow wall-clock.
- Carries **two new persistent state ports** across firings that the old pipeline never needed: **scaffold_species** (in-progress partial assemblies) and **internal_observables** (hook / export-apparatus-subunit / flagella cumulative counts — species with no real bulk-molecule correspondent at all).
- Bridges hook-basal-body completions into real **nascent_flagellum** unique-molecule creation — the same trigger point the old nucleation Step used.

Unchanged, reused as-is: filament-elongation, flgm-secretion, transcription-regulation. They only read/write real bulk molecule counts — they don't care which mechanism produced them. Together they **close the loop**: a completed flagellum triggers FlgM export, freeing FliA to activate Class III transcription, which resynthesizes the very monomers the next NFsim firing will consume.

Mutually exclusive with the old deterministic-Steps pipeline (`flagella_regulation` feature). Opt-in only, for A/B comparison while NFsim's own calibration matures.

Bulk molecule store + persistent Step state
- Free FliC, FlgE / FliD / FlgK / FlgL, FliS, export-apparatus / hook / motor subunits
- **scaffold_species** — partial / "Growing_X" counter-state complexes
- **internal_observables** — export-apparatus-subunit, hook, flagella (no bulk-store home)

↓ **1.** previous firing

↓ **2.** read current bulk counts + carried-over scaffold/internal state → feed to NFsim as observables

FlagellaNFsimComplexation
NFsim / BioNetGen rule-based engine
- Runs its own chunked stochastic update over the interval
- Rules require the prior bond already present on that *same* tracked instance — ordering enforced per-structure, not by exec on order

↓ **3.** hook-basal-body complete → bridges to real **nascent_flagellum** creation

FlagellaFilamentElongation (unchanged)
- Consumes free FliC every tick, extends length on existing nascent_flagellum entries
- On completion: +1 real **CPLX0-7452** bulk count, deletes the unique molecule

↓ **4.** completed flagellum (CPLX0-7452) → triggers FlgM export

FlagellaFlgMSecretion (unchanged)
- n_flagella × rate × dt → exports FlgM (Hughes et al. 1993 calibration)
- Depletes cytoplasmic FlgM, freeing bound FliA

↓ **5.** free FliA rises as FlgM depletes → activates Class III transcription

FlagellaTranscriptionRegulation (unchanged)
- SUM gate over FlhDC + free FliA counts
- Class II promoters (fixed); Class III (fliC, fliD, flgK/L, motAB, fliS, ...) activated as free FliA rises

↓ **6.** new monomers synthesized + structural deltas written back

Bulk store + Step state updated
- Free FliC / subunit counts replenished by transcription, depleted by assembly
- scaffold_species / internal_observables carry forward NFsim's own in-progress state

next tick — NFsim itself only re-fires every ~1200s

Vivarium process execution order — inside the `flagella_nfsim_complexation` feature (`ecoli_baseline.py`)

`tf_binding` → `nfsim-complexation` → `filament-elongation` → `flgm-secretion` → `transcription-regulation` → `transcript_initiation`

Two persistent state ports — why NFsim needs its own memory between firings

Both exist because the Step fires only every ~1200s, not continuously — NFsim's own subprocess is stateless across firings, so anything it needs to remember has to be manually carried forward.

scaffold_species — in-progress partial assemblies

Multi-subunit complexes are modeled as **counter-state scaffold molecules**, not one-shot reactions. Nucleation creates the scaffold; one bimolecular rule per subunit type increments its count each time a free monomer binds; a completion rule converts it to the final complex once every count hits target.

```
Growing_CPLX0_7450(FliF~0..34, FliG~0..34,
FliM~0..34, FliN~0..111)
```

Each `Growing_X` instance is a literal, individually tracked object — two C-rings assembling in parallel are two distinct scaffolds, each with its own independent progress, not one shared bulk count.

Real bug fixed (2026-08-12): the underlying `pbg_nfsim` library gained the capability to carry scaffold state across calls, but nothing wired it to an actual store — every firing silently reset all in-flight progress to zero until this Step's own port closed that loop.

internal_observables — complete species with no bulk address

Three species in the NFsim model correspond to **nothing in the real WCM's bulk store**:

- **flagellar_export_apparatus_subunit** — pure bookkeeping: splits the real 9-reactant export-apparatus assembly into two sequential sub-reactions (the real WCM Step does it in one shot)
- **flagellar_hook** — the real WCM has no standalone "hook complete" molecule at all; the old nucleation Step merged hook-completion and nucleation into one event
- **flagella** — NFsim's "one complete hook-basal-body" observable; in the real pipeline this corresponds to creating a `nascent_flagellum` unique molecule, not a bulk increment

Why it matters most for flagellar_hook: *it's consumed downstream by the flagellum-completion rule. An unconsumed hook finished in one firing must still be available next firing — without this port, every firing would tell NFsim "zero hooks exist," silently erasing real completed material.*

Same problem, two different kinds of state: `scaffold_species` persists *incomplete* structures (still being built, counter-state); `internal_observables` persists *complete* structures that simply have no real-world bulk-molecule address to be written to.

The BioNetGen model, by the numbers

`generate_flagella_bngl.py` → `flagella_complexation.bngl` (verified current — regenerating from the generator reproduces this file exactly)

7	42	28	42	562
assembly reactions	molecule types (35 plain + 7 scaffolds)	seed species (free monomers, t=0)	observables (1 per molecule type)	reaction rule lines (7 nuc + 548 bind + 7 complete)

The 7-stage cascade — subunits consumed per stage

STAGE	SUBUNITS (REAL GENE, COUNT)	SUBUNIT TYPES	TOTAL SUBUNITS	BINDING RULES
flhDC CPLX0-3930	FlhD ×4, FlhC ×2	2	6	4
motor switch C-ring / MS-ring, CPLX0-7450	FliF ×34, FliG ×34, FliM ×34, FliN ×111	4	213	211
export apparatus, rxn 1 subunit intermediate	C-ring ×1, FliO ×1, FliP ×5, FliQ ×4, FliR ×1, FlhA ×9, FliI ×6, FliJ ×1, FlhB ×1	9	29	27
export apparatus, rxn 2 CPLX0-7451	FliH ×12, subunit-intermediate ×1	2	13	11
motor complex FLAGELLAR-MOTOR-COMPLEX	export-apparatus ×1, FliL ×2, FliE ×6, FlgB ×5, FlgC ×6, FlgF ×5, FlgG ×24, FlgH ×26, MotA ×55, MotB ×22	10	152	150
hook flagellar_hook	FlgE ×120	1	120	118
flagellum (final) flagella	FliD ×5, FlgL ×11, FlgK ×11, motor-complex ×1, hook ×1	5	29	27
Total		**33**	**562**	**548**

Why 548 binding rules: the generator unrolls each counter-state increment into its own explicit rule line (`Growing_X(fliG~i) + FliG() → Growing_X(fliG~i+1)` written out for every `i`) rather than one compact range pattern. This is the same combinatorial pressure that got **FliC (5,000 subunits) removed from this model entirely** — at that scale, unrolling one rule per unit was untenable, which is why filament elongation stayed its own separate deterministic Step instead of joining the NFsim network.

Filament elongation mechanics, and a real FliD double-consumption bug — found and fixed

`flagella_filament_elongation.py` (unchanged, shared by both pipelines) — fix applied 2026-08-21 in `generate_flagella_bngl.py`

How elongation actually runs

Runs **every tick** (2s) — not rate-limited like NFsim. Per active `nascent_flagellum` entry (individually tracked: `filament_length` + `massDiff_protein`):

a ≈ 26,450	b ≈ 575	5,000
subunit²/s	subunits	target length

```
rate(L) = a / (b + L) [subunits/s]
```

Renault et al. 2017 (*eLife* 6:e23136) injection-diffusion model — growth slows as the channel gets longer. Target length (5,000) is the real, cited short end of the 20,000–40,000 range, chosen for single-generation tractability (history: 20,000 → 10,000 → 5,000, each a direct-evidence-motivated cut, not arbitrary).

Fair-share: *when several filaments grow simultaneously, if total desired FliC exceeds what's free, every filament is scaled down proportionally — no draw-order bias.*

The nucleation trigger, old vs. new

Under NFsim (this investigation's focus): the bridge in `flagella_nfsim_complexation.py` creates `nascent_flagellum(filament_length=0)` whenever NFsim's `flagella` observable increments.

Under the old deterministic pipeline (`flagella_regulation`, still the default feature): `flagella_filament_nucleation.py` does the same job directly, on a fixed ~600s interval, Poisson-triggered at the real rate **0.00167/s** (Sisti et al. 2017) — concentration-independent.

NFsim's own nucleation is mass-action instead (propensity ∝ [FliF]²) — calibrated to match the real rate only at the reference ambient FliF concentration this investigation has used throughout, a flagged, accepted limitation (see model stats page).

FliD double-consumption

FIXED 2026-08-21

Real biology: FliD assembles into a pentameric cap *once*, before filament elongation begins, and stays at the growing tip for the entire process — "*once the FliD pentamer is completed, it cannot accept another monomer*" (Song et al. 2017, *J Mol Biol* 429:847; corroborated by the cryo-EM mechanism paper, Postel et al. 2020, *Nat Commun* 11:1965). Exactly one consumption event should exist, timed *before* elongation — not after.

The bug: two consumption points fired for the same real flagellum once NFsim was enabled — NFsim's own `flagellum reaction` (5× FliD, at hook-basal-body-cap completion — the biologically correct timing) *and* `flagella_filament_elongation.py`'s completion event (5× FliD, at filament completion — deliberately matching `CPLX0-7452_RXN`'s real coefficient, the **original, shared** design both pipelines depend on). Net effect: 10 FliD consumed per completed flagellum instead of the real 5.

Why the fix belongs on the NFsim side: elongation.py's consumption is the old deterministic pipeline's *only* FliD accounting point (still the default feature) — removing it there breaks that pipeline's mass conservation. NFsim's inclusion of FliD was added independently, cross-referencing real cryo-EM stoichiometry, without checking that the shared downstream Step already covered it.

Fix applied: removed `EG10841-MONOMER[e]: -5.0` from NFsim's `'flagellum reaction'` stoichiometry (`generate_flagella_bngl.py`, old line kept as a comment), regenerated `flagella_complexation.bngl`. Verified directly: FliD no longer appears in the model's real-bulk-ID mapping (31 tracked IDs, down from 32) — confirmed by reimporting the module, not assumed. No ParCa rebuild needed (this file is read fresh from disk at runtime, not baked into the cache).

Not yet re-run: a short NFsim smoke test would confirm the corrected behavior live before trusting it in a longer population run.